

No Single Tool Runs the Pipeline

How Evaluating AI Tools Across a Survival Horror Game's Asset Pipeline Prevents Production Stalls

Preeta Chatterjee | CSYE 7270: Building Virtual Environments | Northeastern University

Topic Claim: *After reading this piece, a practitioner will understand how to evaluate and combine AI tools across a survival horror game's asset and animation pipeline well enough to choose the right tool for each production stage, without assuming any single AI tool handles all creative decisions equally well - and without burning hours on tool-category mismatches that no amount of prompt iteration will fix.*

1. The Scenario

You are three weeks into building Hallway Hunters - a first-person survival horror game set in a zombie-infected apartment complex in Unreal Engine 5. You need five categories of assets before your next submission: a floor plan of the building's layout, a zombie character broken into body parts for rigged 2D animation, a walk cycle animation for that zombie, particle effects for blood splatter and electrical sparks, and Niagara system configuration for those effects. You open an AI tool. Which one?

That question, repeated five times across five different asset types, is the pipeline decision this piece is about. The instinct - especially under a deadline - is to default to one tool and force it to produce everything. That instinct is wrong, and the cost of following it is not a slightly worse output. It is a full production stall: a content filter that refuses your prompt entirely, a flipbook animation that plays as a frozen slideshow, a particle system whose default values produce an effect that looks nothing like what you wanted in your level.

Hallway Hunters is not a large game. It has three floors, one boss, and a rooftop escape. But its asset pipeline crosses at least four distinct categories of generation: spatial documents, character reference art, sequential motion frames, and parametric system configuration. Each of those categories has a different structure of input and output, and each AI tool on the market is optimized for a different one of them. The moment you treat the pipeline as uniform - one tool, one workflow - you guarantee that at least one category will fail.

2. The Mechanism

Understanding why different AI tools fail at different tasks requires understanding what they are actually doing when they generate output. This is not about brand preference. It is about input-output architecture.

Claude

Claude is a language model. When asked to generate an image, it cannot - it produces text that describes an image, or steps that explain how to make one. This is not a failure of Claude. It is a category definition. Claude belongs in the pipeline wherever the output should be language: step-by-step technical instructions, Blueprint logic, parameter suggestions, prompt engineering for other tools. Asking Claude to generate a floor plan image is a category error. Asking Claude how to configure a Niagara particle system is not.

ChatGPT (GPT-4o with image generation)

ChatGPT with image generation produces images from text prompts. Its outputs tend toward photorealism and high detail. For a floor plan prompt, this produces a spatially accurate, readable document - which is exactly what a floor plan needs to be. For a 2D rigged character reference sheet, photorealism becomes a liability: the parts are too detailed, inconsistently styled, and do not fit together visually when assembled into a rig. The tool's strength - high fidelity rendering - is the wrong output for that task.

Gemini

Gemini's image generation produces outputs with a flatter, more illustrative quality. For a 2D rigged zombie character reference sheet, this is an asset rather than a limitation. The parts read as connectable. The style is consistent across body segments. For sequential frame generation - a walk cycle sprite sheet - Gemini fails for the same structural reason every image generation model fails: it generates each image independently. It has no temporal model of motion. Frame 3 does not know what Frame 2 looked like. The result is a set of frames that are visually similar but not sequentially continuous, which means they do not animate.

The structural limit shared by all image generators

No current consumer image generation model - ChatGPT, Gemini, Stable Diffusion, Midjourney - can produce a walk cycle sprite sheet that animates correctly. This is not a prompting problem. The model generates still images. A walk cycle requires frame-to-frame continuity: each frame must be a physically plausible successor to the previous one. Image generators do not have a physics or motion model. They have a statistical model of what images look like. These are different things, and no prompt engineering bridges that gap.

A note on scope: accessibility over capability ceiling

Paid tools such as Midjourney may produce higher-fidelity outputs than the free tiers of ChatGPT and Gemini used in this evaluation. That is not the point. The pipeline documented here was built under the constraint that every tool had to be accessible without a paid subscription — a constraint that applies to the majority of independent developers and students building games outside a funded studio context. Evaluating tools against a capability ceiling that requires payment to reach does not produce a

pipeline that most developers can actually use. The decisions here are made within a realistic access constraint, and the conclusions hold within that constraint. Where paid tools might close a gap — higher image resolution, more consistent style transfer — the gap in question is usually a quality gap, not the structural gap documented in Decision Point 3. No payment tier of any current image generator solves the frame continuity problem, because that problem is architectural, not a feature gating decision.

3. The Design Decision

Across five asset categories in the Hallway Hunters pipeline, each required a separate tool evaluation. The decisions are documented here in the order they occurred during production.

Decision 1: Floor Plan - ChatGPT over Claude

The first prompt went to Claude with a reference image of the existing floor layout: "Using this as a reference, help me create a floor plan for this apartment floor, including the floor plans of the different apartment interiors as well. The space beside apartment 105 is the staircase."

Claude attempted to generate a floor plan and produced a PDF with a dark background that did not correctly follow the reference layout. On the second prompt - specifying white background, black lines - Claude generated an SVG file applying the same spatial logic from the dark background version: a schematic interpretation of the layout rather than an accurate trace of the reference. The SVG was geometrically readable but did not reflect the actual room positions and proportions in the reference image. ChatGPT produced a hand-drawn style floor plan on a white background that accurately reflected the spatial relationships. The decision was clear: ChatGPT for spatial documents requiring reference fidelity.

The lesson is precise: Claude attempted the task across two formats and two prompts, but both outputs were generated from its own interpretation rather than from accurate reading of the reference image. For spatial planning documents where reference accuracy is the requirement, the tool that reads and reflects the reference wins.

Decision 2: Zombie Rig Reference Sheet - Gemini over ChatGPT

The zombie character needed to be broken into isolated body parts - head, torso, arms, legs - laid out on a white background, each part separated and connectable for rigged sprite animation. Two prompts were sent to ChatGPT:

```
Prompt 1: zombie body parts reference sheet, side view facing left,
2D flat illustration, all parts separated with clear space between
them, laid out on white background: head, torso, right arm, left
arm, right leg, left leg, each part isolated with no overlapping,
```

muted gray-green horror palette, tattered clothing, simple clean linework, no shadows, anatomical proportion, centered on canvas

Prompt 2: [same as above] - parts should be connectable later for rigged sprite animation

ChatGPT produced parts that were too realistic in rendering style and did not visually cohere as a rig - the shading, line weight, and proportions varied between segments in ways that would be obvious when assembled. Gemini produced a flat 2D illustration style with consistent linework across all parts. The parts fit together visually. The decision was to use Gemini for 2D character reference work where stylistic consistency across segments mattered more than rendering fidelity.

Decision 3: Walk Cycle - Rig over Flipbook (and why AI cannot close the gap)

The animation assignment required a walk cycle. The initial approach was to generate a flipbook sprite sheet using AI - a horizontal row of frames showing the zombie moving through a complete walk cycle. Multiple prompts were sent to both ChatGPT and Gemini across several iterations. A representative final prompt:

8-frame zombie walk cycle sprite sheet, side view facing left, 2D flat illustration style, dark horror aesthetic, decayed skin, tattered clothing, each frame should show a very distinct walk pose, white background, no shadows, no overlapping frames, evenly spaced in a single horizontal row, simple clean linework, muted grey-green colour palette, all frames should be unique. arms and legs should complete walk cycle in those 8 movements for flipbook animation

Neither tool produced usable output. The frames were not distinct enough from each other - some were virtually identical - and playing them in sequence produced no coherent motion. The walk cycle did not complete. The decision at this point was not to iterate further on the prompt. It was to switch methods entirely.

The alternative was a rigged sprite animation: taking the Gemini-generated body part reference sheet, importing the parts into Unity, assigning pivot points manually, and animating each keyframe by hand. This approach produces a walk cycle where every timing, weight shift, and limb position is under explicit human control. The sample rate is adjustable. The feel - whether the zombie moves with a lurching, dragging quality or a faster shamle - is a parameter, not a fixed output.

The flipbook approach, even with perfectly generated frames, would have locked those decisions into the image. The rig approach keeps them open. This is the design decision: not which AI tool to use, but when to exit the AI pipeline entirely and use human animation control instead.

Decision 4: Blood Splatter Niagara System - Claude's structure, human material

For the blood splatter particle effect triggered when the player or a zombie is struck, Claude was asked for a step-by-step Niagara configuration. Claude produced a complete sequence covering emitter setup, spawn rate, lifetime, velocity, gravity, drag, color, and material. The Niagara system parameters - burst count, velocity cone, gravity, drag - were followed as suggested. The material was rebuilt entirely from scratch.

Claude suggested a material using Translucent blend mode and Unlit shading, with the texture connected to Emissive Color for guaranteed visibility in dark environments, using a simple radial gradient as the sprite shape. Every one of those decisions was reversed in the final implementation.

The final material uses Masked blend mode and Default Lit shading. Masked means the material is either fully opaque or fully transparent at each pixel - no soft feathering. Default Lit means it receives scene lighting rather than emitting its own. These two changes together mean the blood splatter reads as a physical object sitting in the level rather than a glowing translucent overlay. Removing the Emissive Color connection removes the glow entirely. The splatter is matte.

The opacity chain is the furthest departure. Instead of a radial gradient, the final material uses: Texture Coordinate scaled by 0.3, fed into a Panner at X 0.1 Y 0.2, into a Perlin noise texture, through a Smoothstep at 0.3 to 1.0, multiplied with the texture alpha, then multiplied again with Particle Color alpha into the Opacity Mask. This chain gives each sprite an organically irregular edge driven by noise rather than a clean soft circle. The splatter looks like blood, not like a particle.

The structural steps Claude provided - how to set up the emitter, configure burst behavior, route the Blueprint trigger - were correct and implemented as suggested. The material philosophy was a complete departure, informed by a YouTube tutorial by Surin combined with the judgment that a matte, lit, noise-edged result would read more credibly in a horror environment than a glowing translucent blob. This combination - AI for structure, human for visual philosophy - is the pattern the pipeline settled into across multiple effects.

4. The Failure Case

The clearest failure in this pipeline is the walk cycle flipbook. It is worth examining mechanistically because it illustrates the structural limit that no prompt revision can overcome.

Image generation models produce outputs by sampling from a learned distribution of what images look like. They do not simulate physics. They do not model motion trajectories. When asked to produce Frame 3 of a walk cycle, the model has no knowledge of what Frame 2 contained. It produces a plausible-looking pose independently. When all eight frames are assembled and played in sequence, the result is not a walk cycle. It is a sequence of eight unrelated poses. The zombie's center of mass shifts arbitrarily between frames. The foot contact points are inconsistent. The arm swing does not correspond to any leg position.

This was tested across multiple iterations on both ChatGPT and Gemini. The prompts became progressively more specific - "each frame should show a very distinct walk pose," "arms and legs should complete the walk cycle in those 8 movements" - and the outputs did not improve in the dimension that mattered. The frames became slightly more distinct from each other, but they remained sequentially incoherent. Playing them produced a choppy, stuttering result that did not read as motion.

The failure is not a prompting failure. It is an architectural one. The tool cannot produce what the task requires because what the task requires - temporal continuity across frames - is not in the model's output space. Recognizing this early prevents the most expensive mistake in AI-assisted production: continuing to iterate on a prompt when the failure is structural.

A second, subtler failure appears in the Niagara parameter case. Claude's suggested values were not wrong in any absolute sense - a spawn rate of 15 particles per second, a sprite size of 3 units, a base light intensity of 800 are all reasonable defaults. They were wrong for this specific level, at this specific viewing distance, with this specific lighting environment. An AI generating configuration parameters has no perceptual access to the scene it is configuring. The values it produces are averages drawn from documentation and tutorials. They will be correct on average and wrong in specifics. The human in the loop is not a quality check on the AI output. The human is the only agent with access to the information required to make the final call.

5. The Exercise

This exercise is designed to reproduce the walk cycle failure in under fifteen minutes and to make the structural limit visible rather than abstract.

Step 1. Open ChatGPT or Gemini. Send this prompt exactly:

```
4-frame zombie walk cycle sprite sheet, side view facing left, 2D  
flat illustration, white background, each frame shows a distinct  
walk pose, frames evenly spaced in a horizontal row, arms and legs  
complete walk cycle in 4 movements
```

Step 2. Save the output image. Import it into any tool that can play sprite sheet frames - Unity's sprite animator, a simple Photoshop frame animation, or even a PowerPoint slide transition set to 0.1 seconds per frame.

Step 3. Play the animation. Observe whether it reads as continuous motion or as a sequence of static poses.

Step 4. Now send the same prompt to the other tool (if you used ChatGPT, send to Gemini, and vice versa). Compare the outputs. Note whether either produced usable sequential motion.

Step 5. Now identify one asset in your own project that would survive this failure - a texture, a UI element, a static prop - and one asset that would not. The assets that would not survive are the ones where you need either hand animation or a dedicated

motion tool. Document the distinction. That documentation is your tool evaluation for your pipeline.

The variation that breaks the system most clearly: ask for a 12-frame walk cycle instead of 4. The increased frame count makes the incoherence more visible - the model's inability to maintain continuity compounds across more frames. The output will be even less usable than the 4-frame version, despite being more specific in its prompt.

Human Decision Node

MANDATORY HUMAN DECISION NODE

Context: Blood splatter particle material for NS_BloodSplatter in Hallway Hunters

AI proposed (Claude): Translucent blend mode, Unlit shading, Emissive Color

connection, radial gradient for sprite opacity shape.

I rejected because:

Translucent + Unlit + Emissive = a glowing soft blob. Blood does not glow.

In a horror environment, a matte physical surface reads as real.

A glowing translucent overlay reads as a UI element or a magic effect.

My decision:

Blend Mode: Masked (fully opaque or fully transparent per pixel, no feather)

Shading Model: Default Lit (receives scene lighting, no self-emission)

Emissive Color: not connected (no glow)

Opacity chain: TexCoord scaled 0.3 → Panner X0.1 Y0.2 → Perlin noise →

Smoothstep 0.3-1.0 → multiply texture alpha → multiply Particle Color

alpha → Opacity Mask

Result: organically irregular noise-driven edge instead of soft radial circle.

Source: Surin YouTube tutorial + perceptual judgment in level environment.

The Niagara system parameters (burst, velocity, gravity, drag) followed

Claude's suggestions. The material is a complete rebuild.

Conclusion: AI Is a Pipeline Decision

The master claim of this course is that AI is a pipeline decision, not a magic layer. Where you put it - and how you constrain it - determines what your game becomes. The Hallway Hunters asset pipeline is a specific instance of that claim.

Claude belongs in the pipeline for language: technical steps, Blueprint logic, parameter suggestions, prompt generation for other tools. It does not belong in the pipeline for image generation because it cannot produce images. ChatGPT belongs for spatial documents and reference art requiring high fidelity. Gemini belongs for 2D character reference requiring stylistic consistency across parts. Neither belongs for walk cycle animation - not because of any deficiency in their image generation, but because sequential frame continuity is structurally outside the output space of image generation models. Niagara parameter configuration belongs to the AI for structural steps and to the human for perceptual tuning.

None of these decisions were obvious in advance. They were made by running each tool at its actual task, observing where the output structure matched the task's requirements, and switching when it did not. That process - evaluate, observe, switch, document - is the pipeline evaluation practice this piece is arguing for. The alternative is defaulting to one tool, iterating on prompts that cannot fix structural mismatches, and stalling production on a deadline that does not move.

The game ships or it does not. The tool evaluation is what makes the difference.